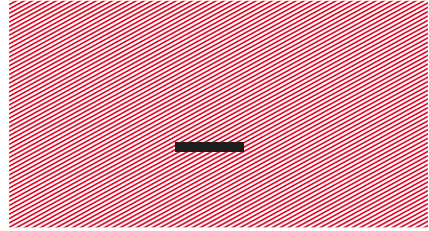


— IoT Devices

Les 5 – SPI





SPI

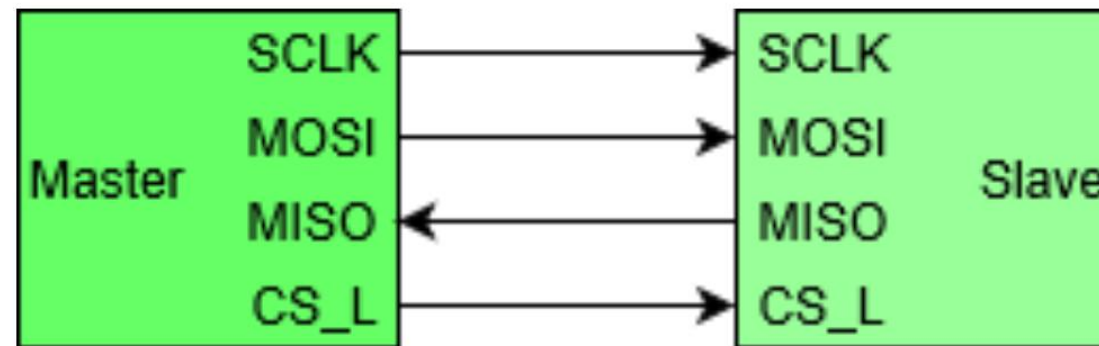
algemene info



- SPI = **Serial Peripheral Interface**
- Een communicatiebus dus, die data **serieel verstuurt** (via één draad).
- UART is ook serieel, maar asynchroon dus zonder klok... Daarom moet je bij een UART steeds een baudrate 'afspraken'.
- SPI is echter een **synchrone bus**. Ze heeft dus een kloksignaal! Je hoeft geen 'baudrate' af te spreken. Het **ritme van de klok** bepaalt het ritme van de datatransfert.
- Wordt typisch gebruikt om te communiceren **tussen IC's op éénzelfde print...**

- Veel gebruikte **naamgevingen**:
 - MOSI = Master Output Slave Input
 - MISO = Master Input Slave Output
 - (N)SS = Slave Select
 - SCK = Serial Clock
 - Mogelijke alternatieven:
 - SDO = Serial Data Out
 - SDI = Serial Data In
 - CS = Chip Select
- PICO = Peripheral In Controller Out
POCI = Peripheral Out Controller In

Verschillende **typologieën** (soorten opstellingen)

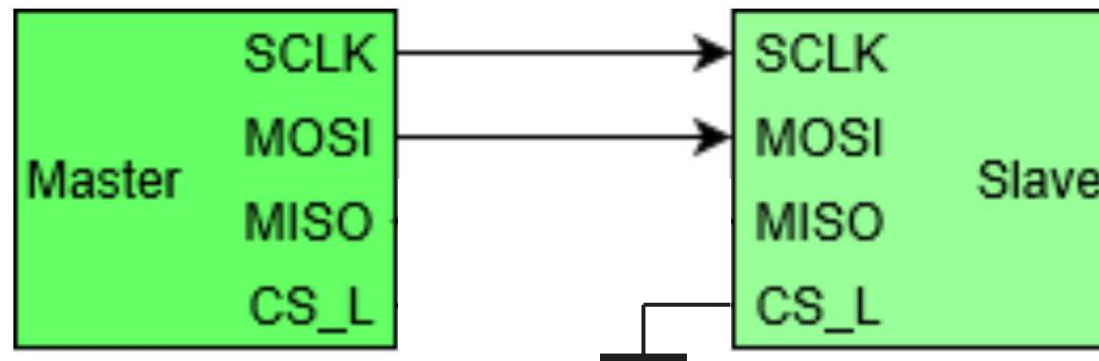


SPI Bus – Simple Point-to-Point
Topology

Bron: <https://practicalee.com/spi>

SPI algemene info

Verschillende **typologieën** (soorten opstellingen)



SPI Bus – Simple Point-to-Point
Topology

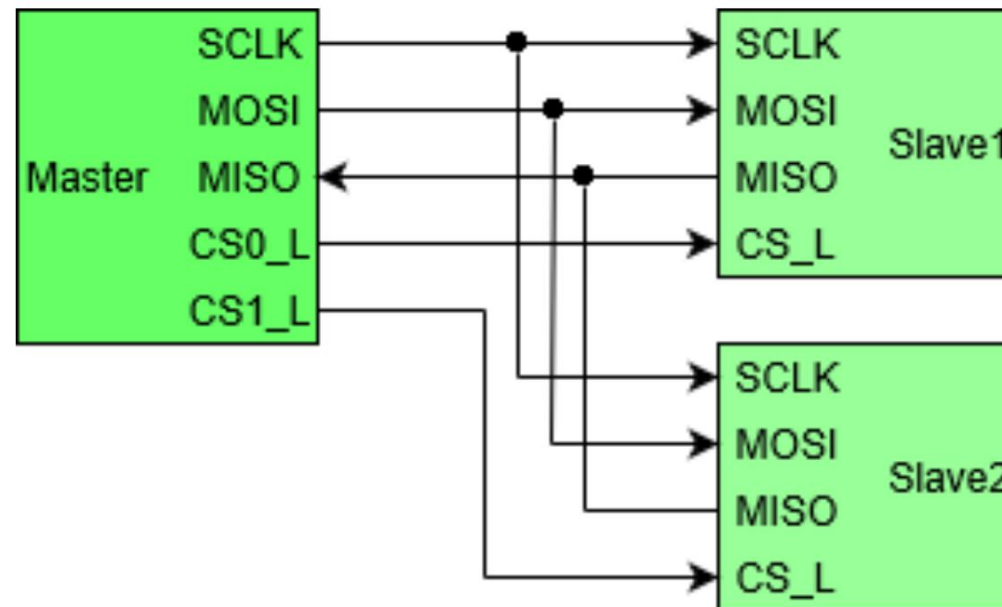
OPM: je kan in sommige gevallen ook enkel data van master naar slave sturen en de chip select constant laag houden...

Sommige IC's hebben zelfs geen chip select (APA102C RGB LED).

Bron: <https://practiclee.com/spi>

SPI algemene info

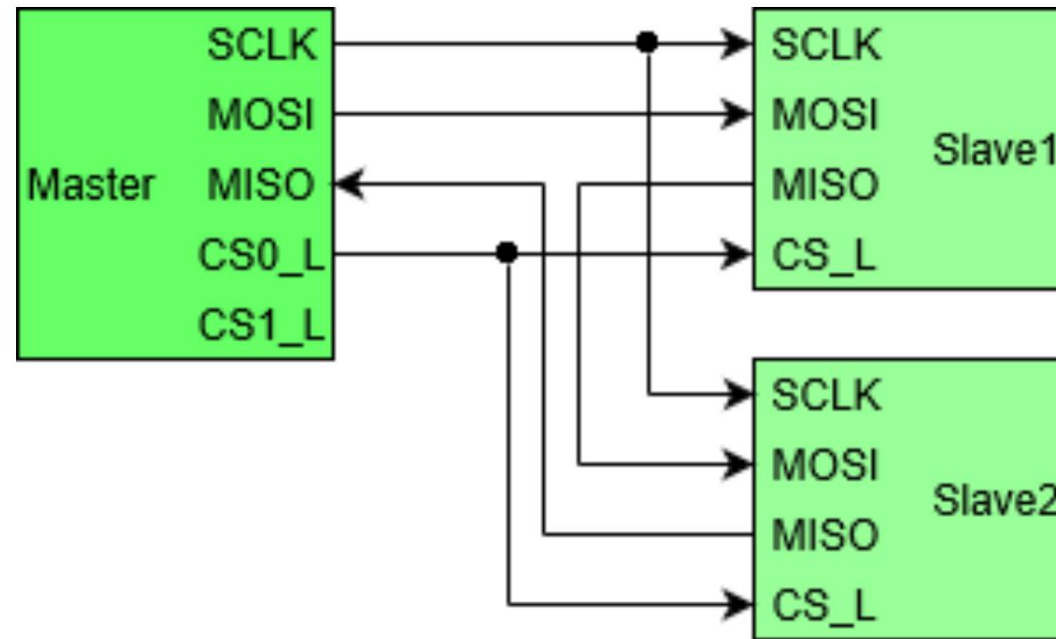
Verschillende **typologieën** (soorten opstellingen)



SPI Bus – Star Topology

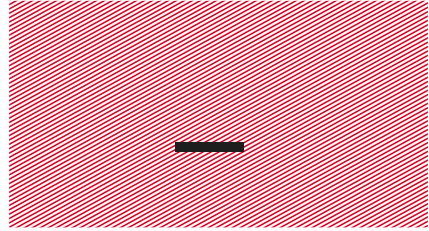
Bron: <https://practicalee.com/spi>

Verschillende **typologieën** (soorten opstellingen)



SPI Bus – Daisy Chain Topology

Bron: <https://practicalee.com/spi>



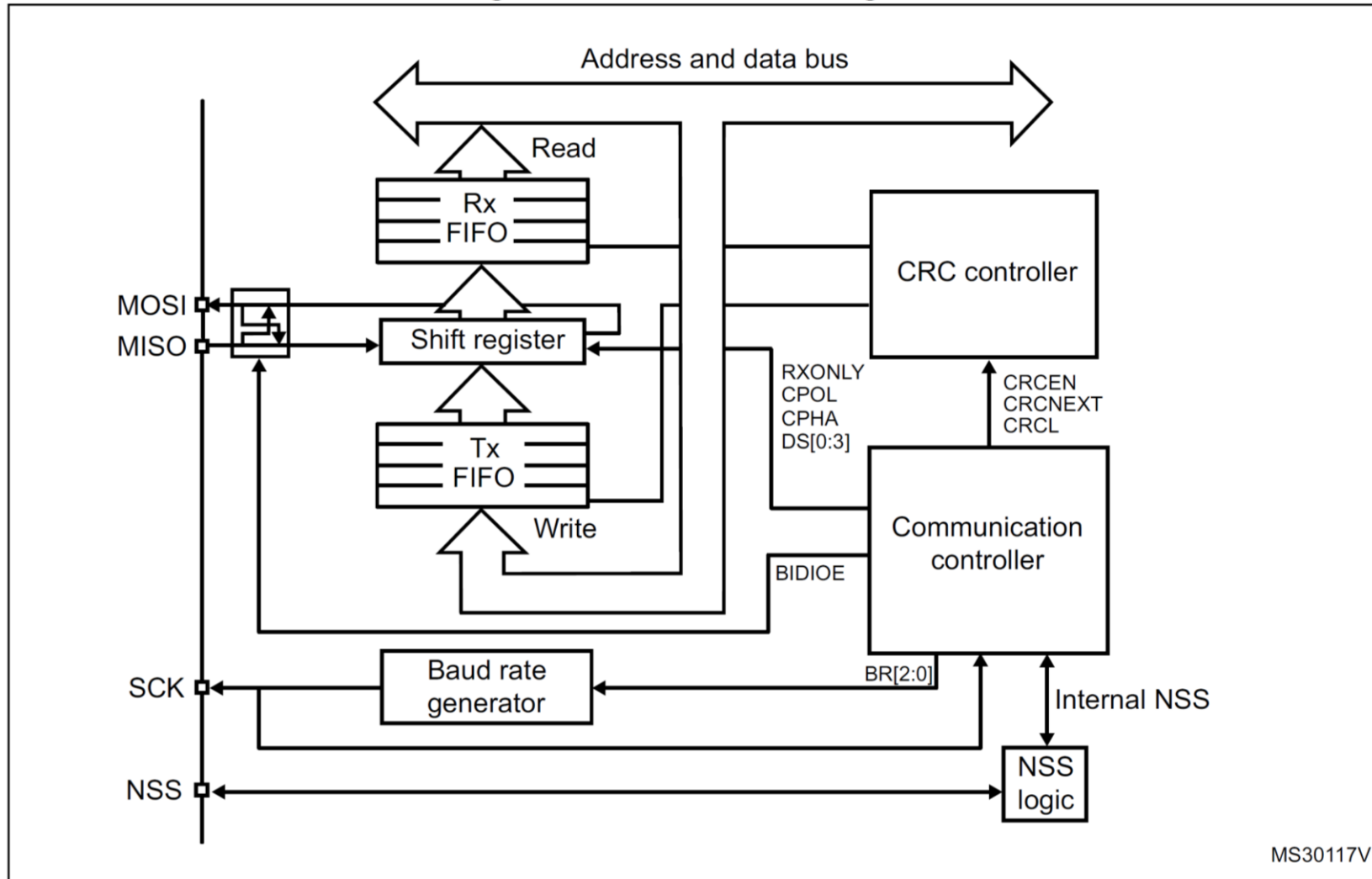
SPI

in de STM32L476RG



SPI in de STM32L476RG

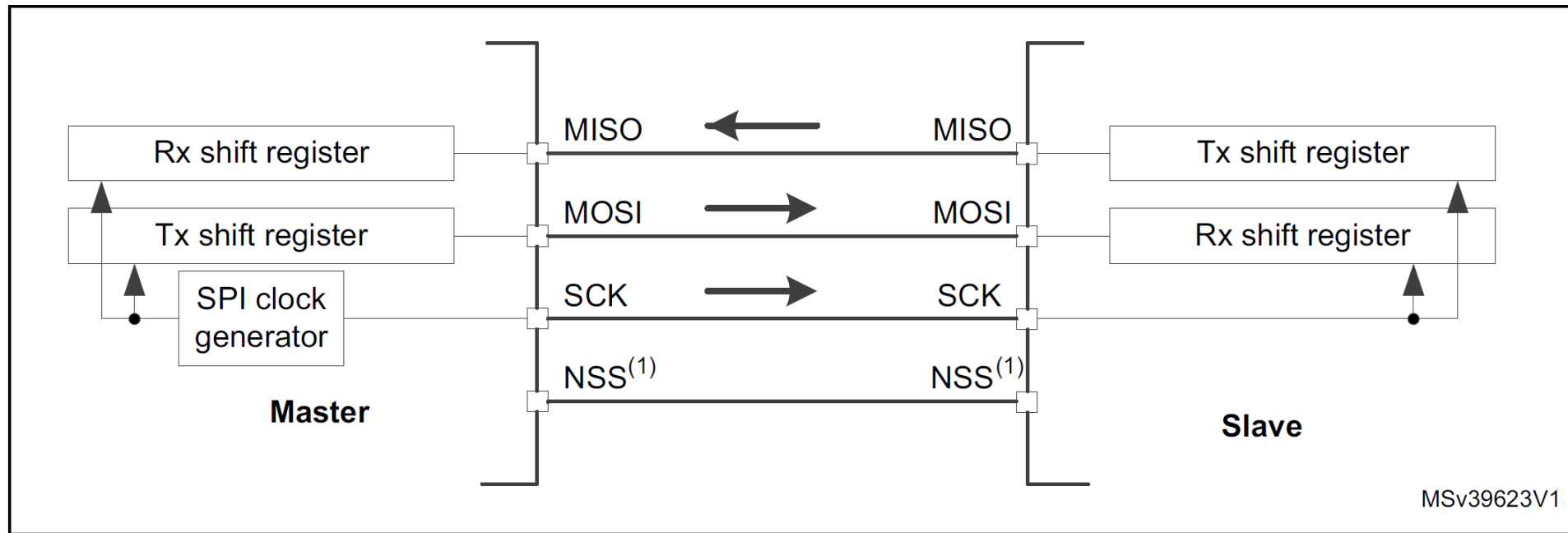
Figure 456. SPI block diagram



MS30117V1

SPI in de STM32L476RG

Figure 457. Full-duplex single master/ single slave application

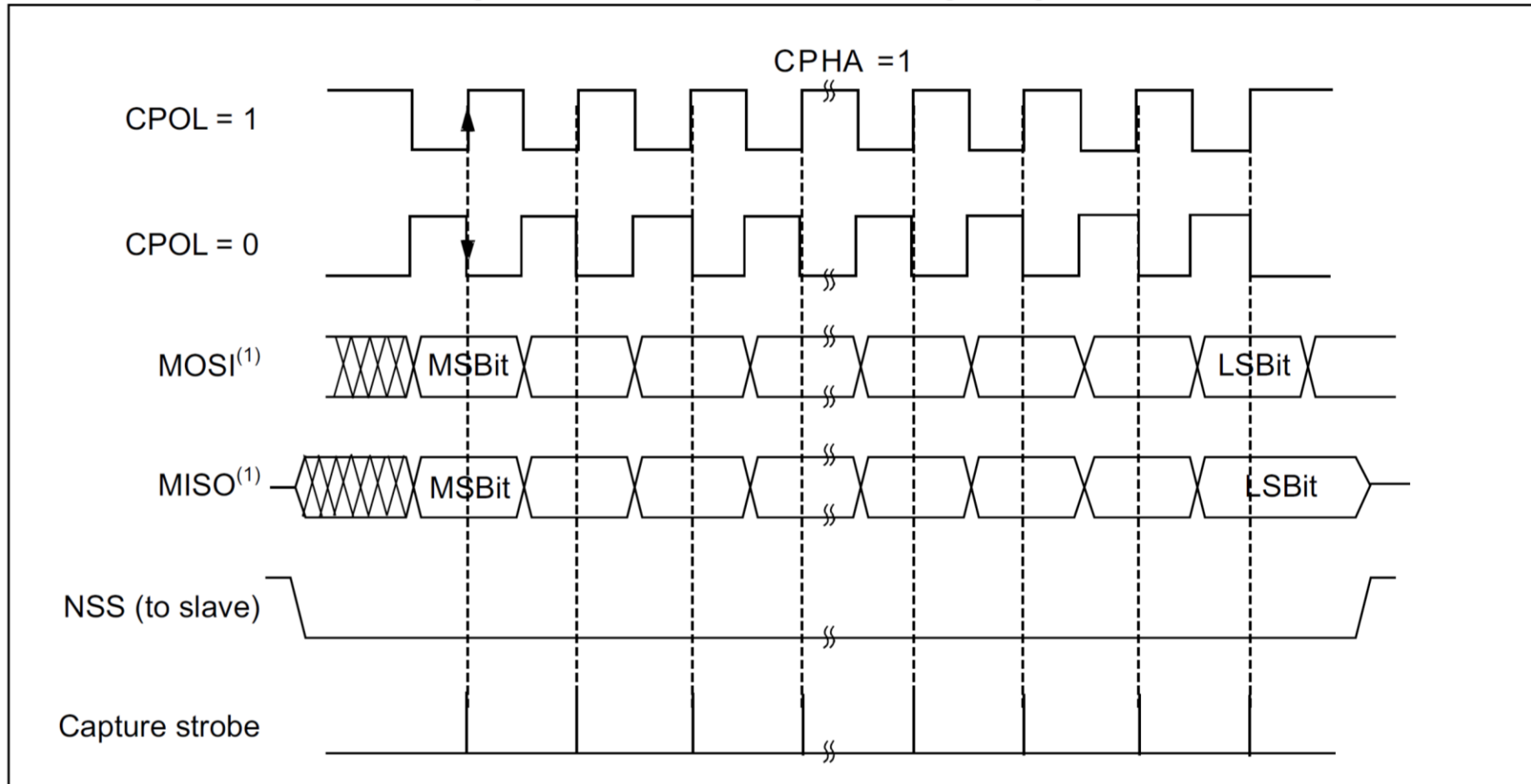


1. The NSS pins can be used to provide a hardware control flow between master and slave. Optionally, the pins can be left unused by the peripheral. Then the flow has to be handled internally for both master and slave. For more details see [Section 42.4.5: Slave select \(NSS\) pin management](#).

- SPI kan in 4 verschillende modes communiceren. Die mode wordt ingesteld met de 'clock polarity' (CPOL) en 'clock phase' (CPHA).
- Als CPOL nul is, dan is de het kloksignaal laag indien er geen data verstuurd wordt. Als CPOL één is, dan is het kloksignaal hoog indien er geen data verstuurd wordt.
- Als CPHA nul is, wordt de eerste flank (stijgend of dalend) van de klok gebruikt om data in te kloppen. Als CPHA één is, wordt de tweede flank van de klok gebruikt om data in te kloppen.
- Zie afbeelding volgende slides...

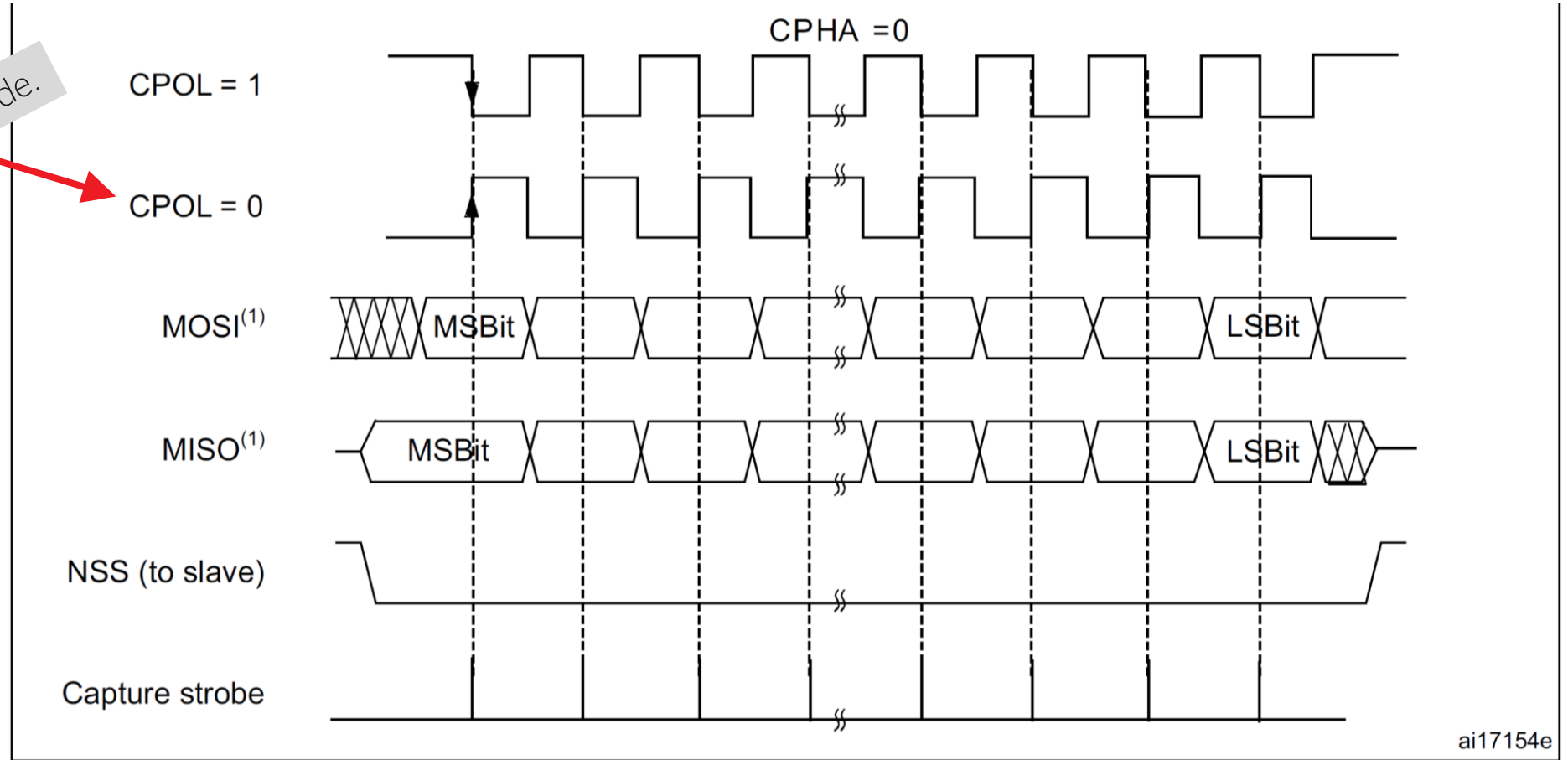
SPI in de STM32L476RG

Figure 463. Data clock timing diagram



SPI in de STM32L476RG

Veel gebruikte mode.



SPI in de STM32L476RG

- CPOL en CPHA kan je instellen in het SPIx->CR1 register.

42.6.1 SPI control register 1 (SPIx_CR1)

Address offset: 0x00

Reset value: 0x0000

Waarvoor zou BR[2:0] dienen? ;-)

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BIDI MODE	BIDIOE	CRC EN	CRCN EXT	CRCL	RX ONLY	SSM	SSI	LSB FIRST	SPE	BR[2:0]			MSTR	CPOL	CPHA
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 1 **CPOL**: Clock polarity

0: CK to 0 when idle

1: CK to 1 when idle

Note: This bit should not be changed when communication is ongoing.

This bit is not used in SPI TI mode except the case when CRC is applied at TI mode.

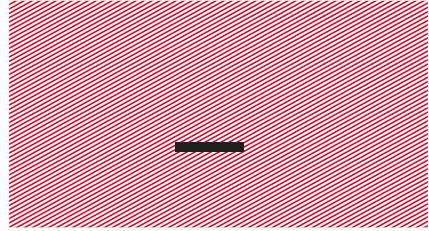
Bit 0 **CPHA**: Clock phase

0: The first clock transition is the first data capture edge

1: The second clock transition is the first data capture edge

Note: This bit should not be changed when communication is ongoing.

This bit is not used in SPI TI mode except the case when CRC is applied at TI mode.

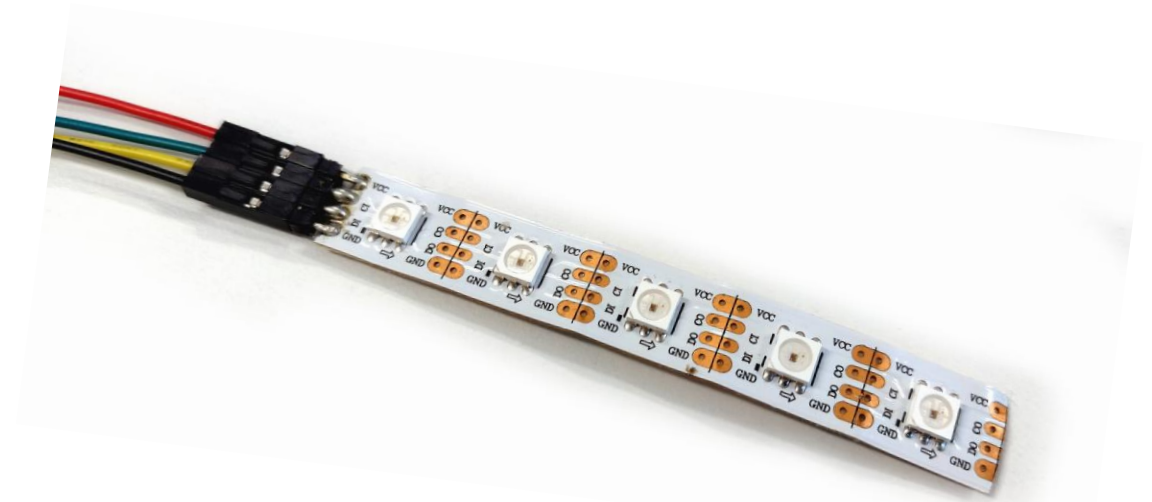
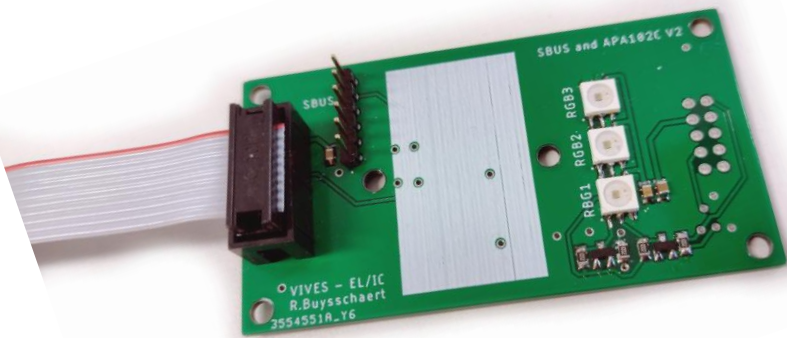


APA102C



— APA102C

- De APA102C is een **RGB LED** die aanstuurbaar is via SPI.
- Je kan er één aparte LED mee aansturen of een volledige LED-strip.
- Werkt met een '**start frame**' en een '**end frame**' (zie datasheet).



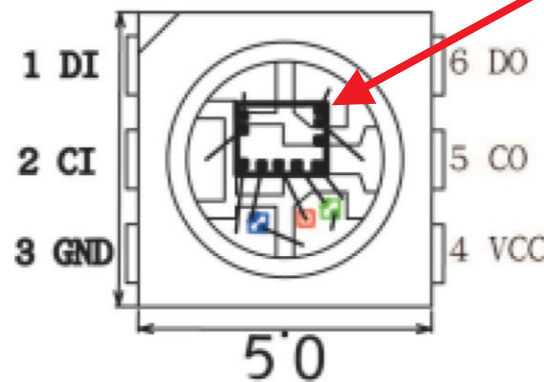


APA102C

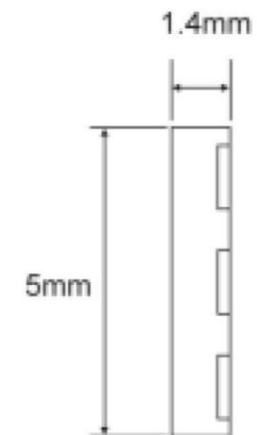
- In ieder LED zit een **chip**, die de SPI-communicatie en LED-sturing op zich neemt.
- De **uitgang** van de ene LED, kan **gekoppeld** worden met de **ingang** van de andere LED.

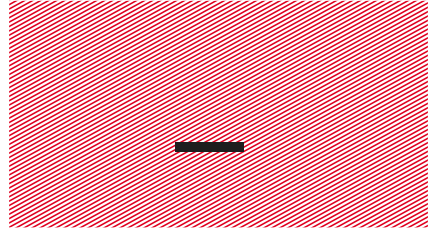


PHYSICAL DIMENSIONS



DI	: Data input
CI	: Clock input
DO	: Data output
CO	: Clock output
GND	: -
VCC	: + 5V





APA102C aansturen via polling

APA102C aansturen via polling

- De **eenvoudigste** manier om de LED's aan te sturen is via blokkerende functies, via polling dus.

HAL_SPI_Transmit

Function name

HAL_StatusTypeDef HAL_SPI_Transmit (SPI_HandleTypeDef * hspi, uint8_t * pData, uint16_t Size, uint32_t Timeout)

Function description

Transmit an amount of data in blocking mode.

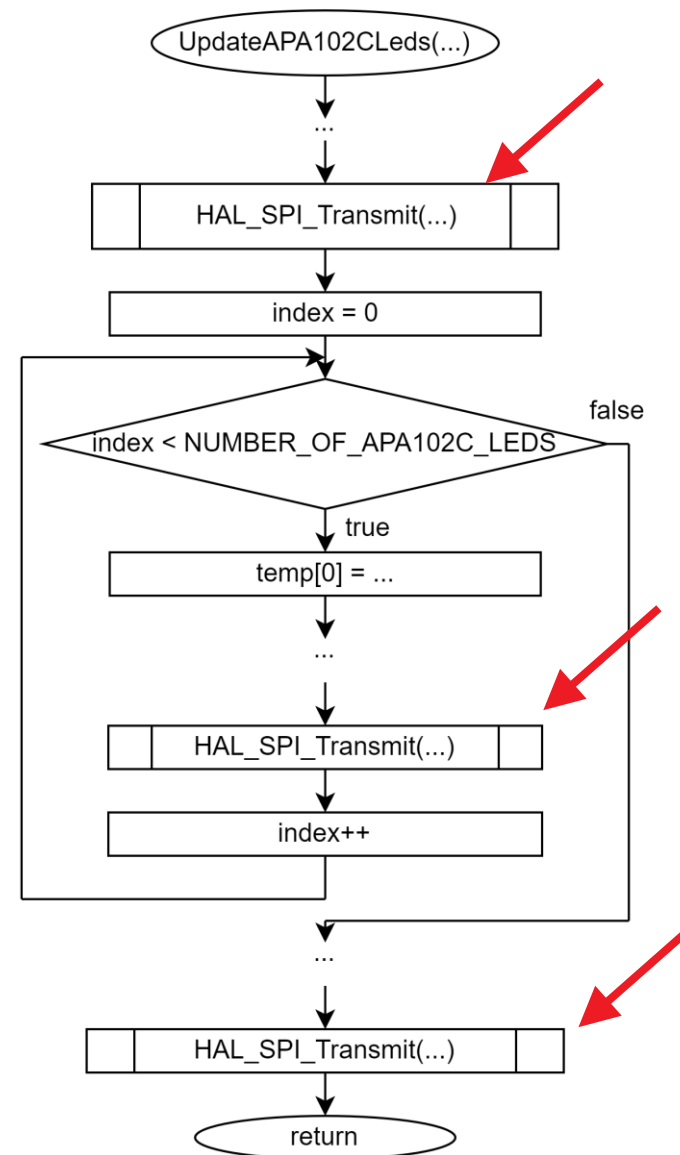
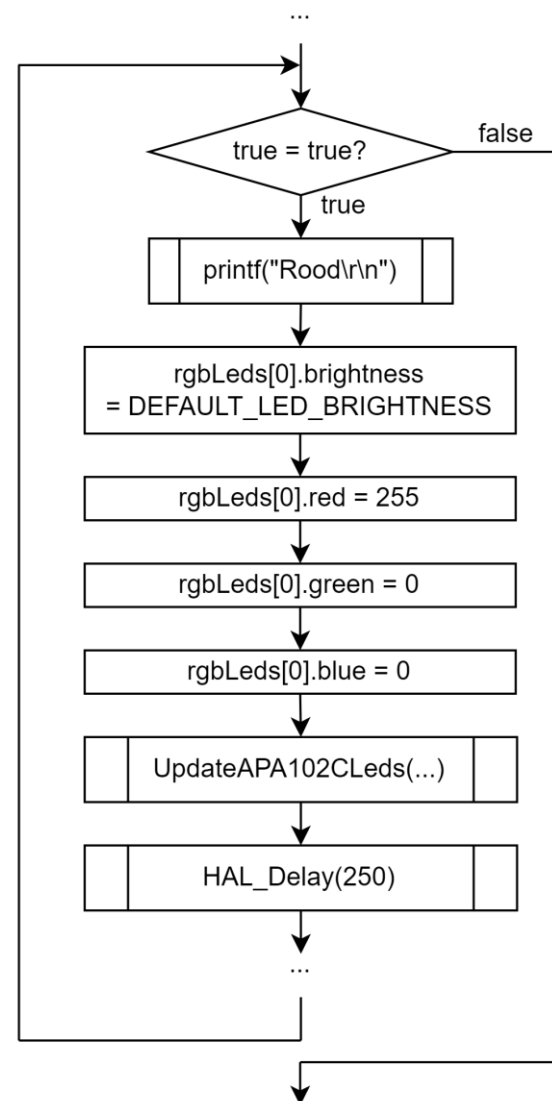
Parameters

- hspi**: pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.
- pData**: pointer to data buffer
- Size**: amount of data to be sent
- Timeout**: Timeout duration

Return values

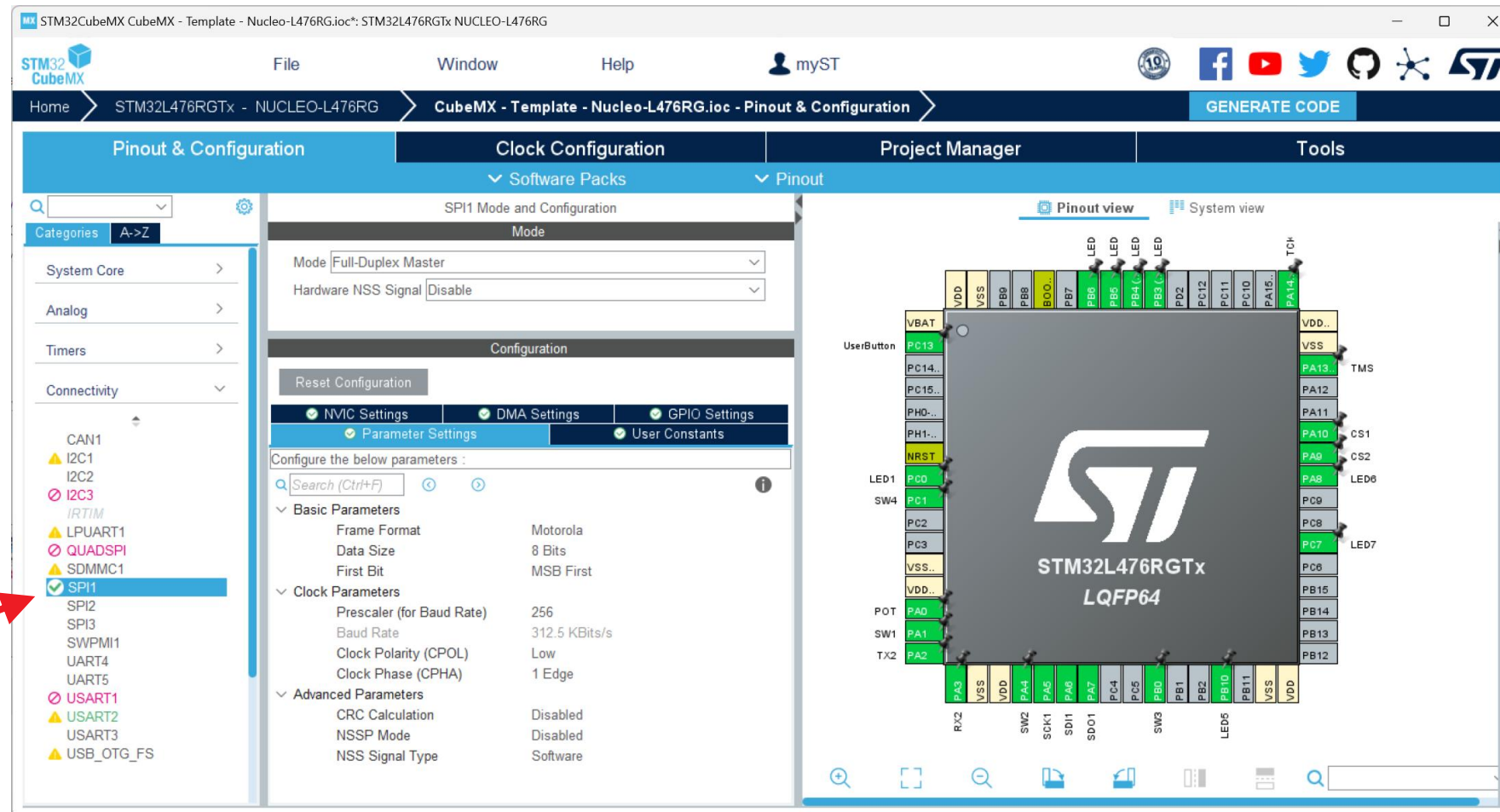
- HAL**: status

APA102C aansturen via polling



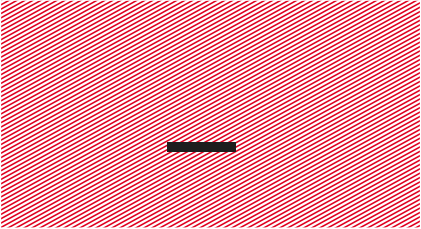
APA102C aansturen via polling

- Stel de SPI-module in via STM32CubeMX.



— APA102C aansturen via polling

```
1 #include "stm32l4xx_hal.h"
2 #include "apa102c.h"
3
4 // Verstuur de data over SPI1. Let op: zorg dat SPI1 reeds geïnitialiseerd is.
5 void UpdateAPA102CLeds(SPI_HandleTypeDef* hspi, APA102C led[])
6 {
7     uint8_t index = 0;
8     uint8_t temp[] = {0, 0, 0, 0};
9
10    HAL_SPI_Transmit(hspi, temp, sizeof(temp), 100);    // Start frame.
11
12    for(index = 0; index < NUMBER_OF_APA102C_LEDS; index++)
13    {
14        temp[0] = led[index].brightness | 0xE0;        // 5-bit intensiteit.
15        temp[1] = led[index].blue;                    // Blauw.
16        temp[2] = led[index].green;                  // Groen.
17        temp[3] = led[index].red;                    // Rood.
18
19        HAL_SPI_Transmit(hspi, temp, sizeof(temp), 100);
20    }
21
22    temp[0] = 255;
23    temp[1] = 255;
24    temp[2] = 255;
25    temp[3] = 255;
26    HAL_SPI_Transmit(hspi, temp, sizeof(temp), 100);    // End frame.
27 }
```



APA102C aansturen via DMA

APA102C aansturen via DMA

- De **beste** manier om de LED's aan te sturen is via DMA-functies, niet blokkerende functies dus.

HAL_SPI_Transmit_DMA

Function name

HAL_StatusTypeDef HAL_SPI_Transmit_DMA (SPI_HandleTypeDef * hspi, uint8_t * pData, uint16_t Size)

Function description

Transmit an amount of data in non-blocking mode with DMA.

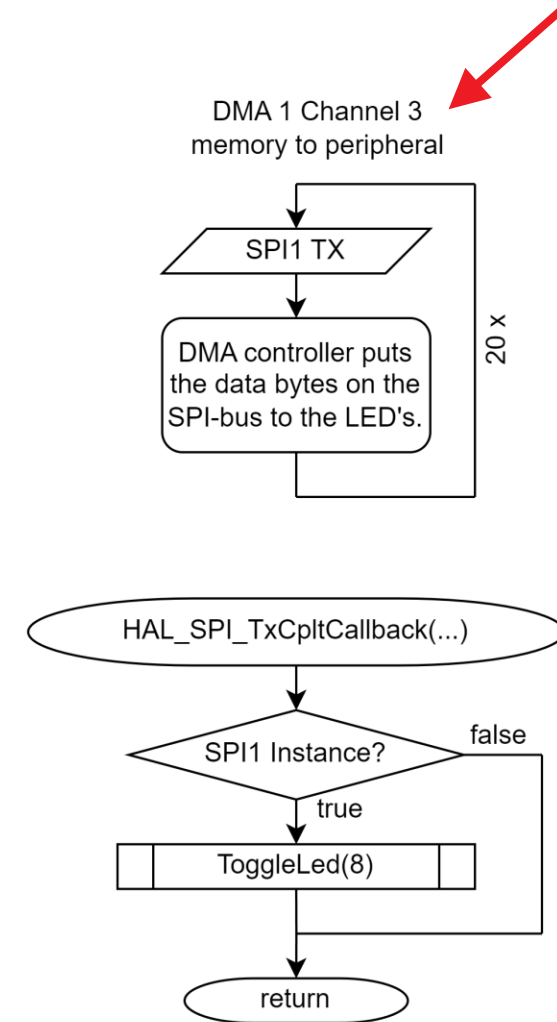
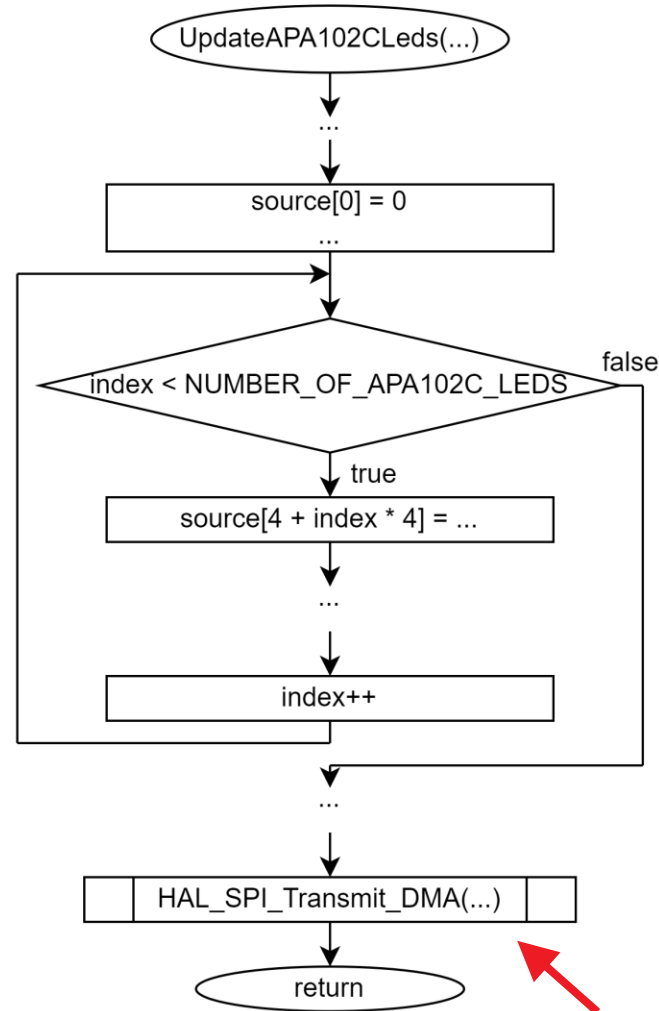
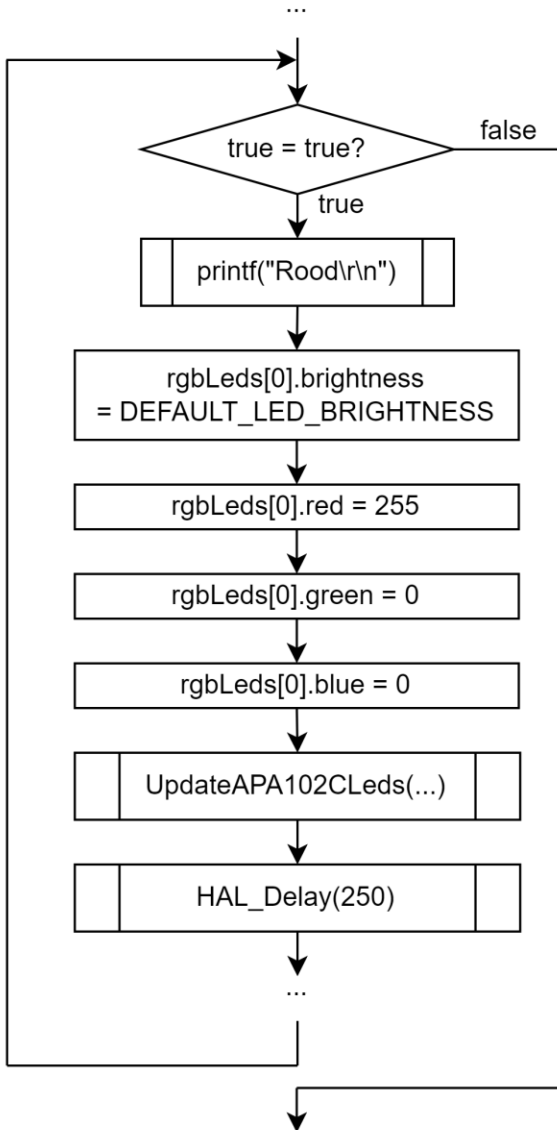
Parameters

- hspi**: pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.
- pData**: pointer to data buffer
- Size**: amount of data to be sent

Return values

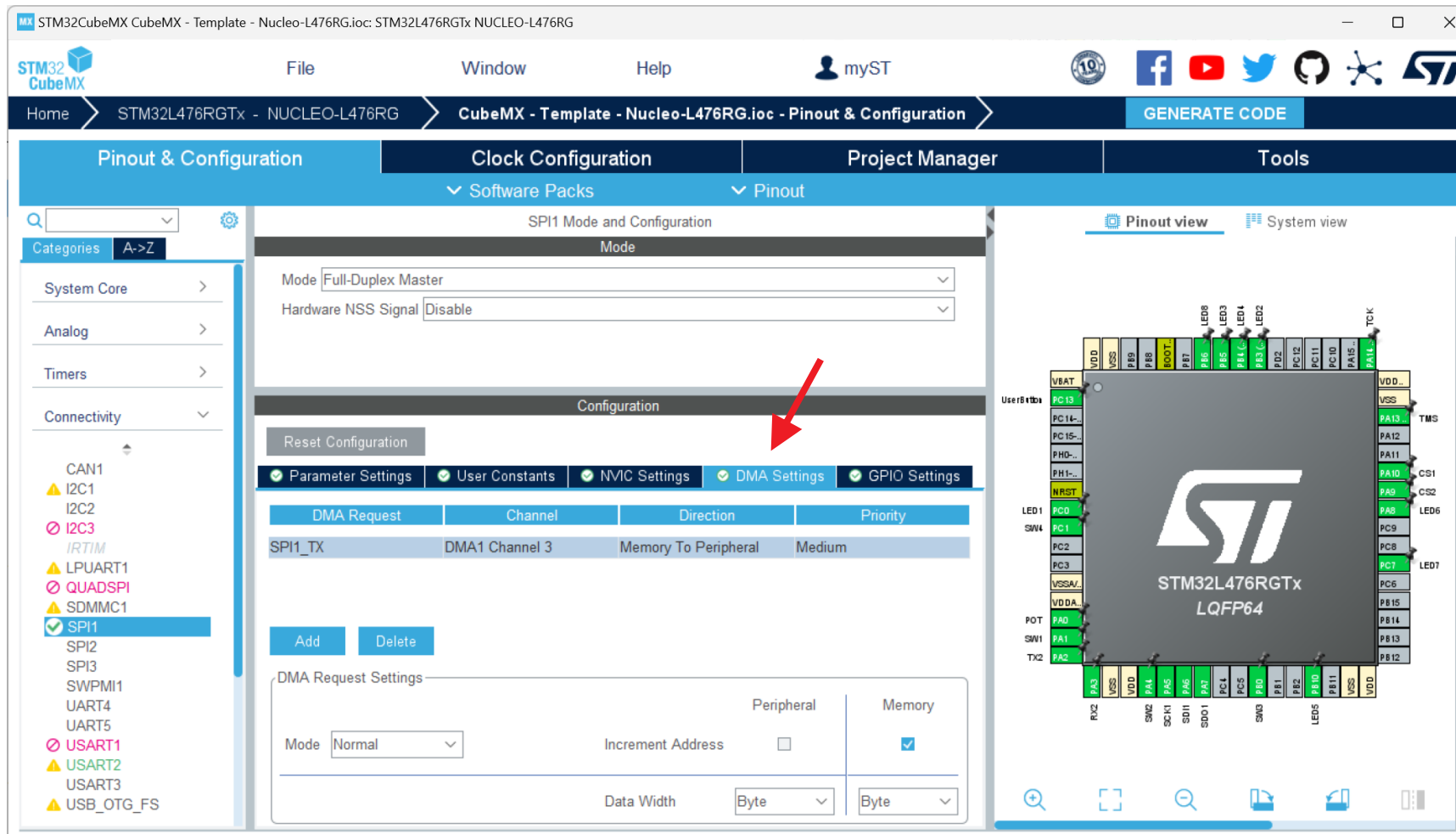
- HAL**: status

APA102C aansturen via DMA



APA102C aansturen via DMA

- Stel de SPI-module in via STM32CubeMX.

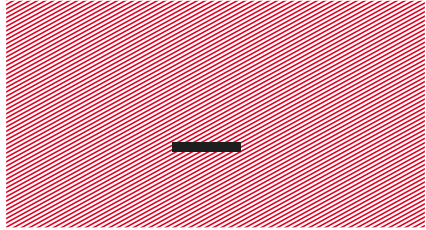


APA102C aansturen via DMA

```
1 #include "stm32l4xx_hal.h"
2 #include "apa102c.h"
3
4 // Verstuur de data over SPI1. Let op: zorg dat SPI1 reeds geïntialiseerd is.
5 void UpdateAPA102CLeds(SPI_HandleTypeDef* hspi, APA102C led[])
6 {
7     uint8_t index = 0;
8     extern uint8_t source[4 + NUMBER_OF_APA102C_LEDS * 4 + 4];
9
10    source[0] = 0;    // Start frame.
11    source[1] = 0;
12    source[2] = 0;
13    source[3] = 0;
14
15    for(index = 0; index < NUMBER_OF_APA102C_LEDS; index++)
16    {
17        source[4 + index * 4] = led[index].brightness | 0xE0;    // 5-bit intensiteit.
18        source[5 + index * 4] = led[index].blue;                // Blauw.
19        source[6 + index * 4] = led[index].green;               // Groen.
20        source[7 + index * 4] = led[index].red;                 // Rood.
21    }
22
23    source[4 + NUMBER_OF_APA102C_LEDS * 4] = 255;    // End frame.
24    source[5 + NUMBER_OF_APA102C_LEDS * 4] = 255;
25    source[6 + NUMBER_OF_APA102C_LEDS * 4] = 255;
26    source[7 + NUMBER_OF_APA102C_LEDS * 4] = 255;
27
28    // Alle data verzenden via DMA.
29    HAL_SPI_Transmit_DMA(hspi, source, (4 + NUMBER_OF_APA102C_LEDS * 4 + 4));
30 }
```

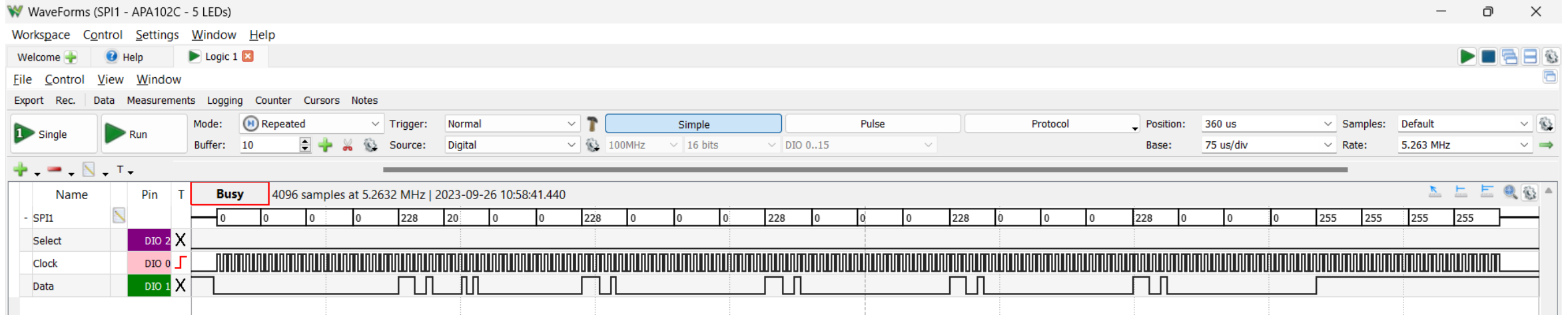
APA102C aansturen via DMA

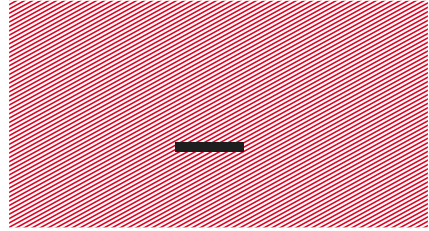
```
597 /* USER CODE BEGIN 4 */
598 void HAL_SPI_TxCpltCallback(SPI_HandleTypeDef * hspi)
599 {
600     if(hspi->Instance == SPI1)
601     {
602         ToggleLed(8); // Teken van leven geven...
603     }
604 }
605 /* USER CODE END 4 */
```



APA102C aansturen via DMA

- Hieronder het **scoopbeeld** voor het aansturen van **5** APA102C LED's.
- Kan je het '**protocol**' erin terugvinden?





Wheel LED's





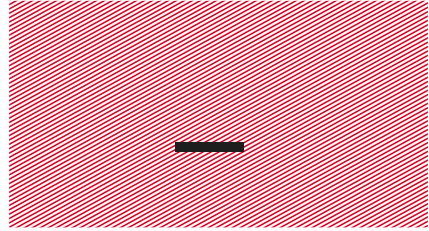
Wheel LED's

Doel? Een fietswiel gebruiken als 'ronddraaiend' beeldscherm van 500 x 16 pixels.

Hoe? Via adresseerbare APA102C LED's, SPI met DMA, een power bank en een Nucleo-L476RG.

Resultaat? Zie demo...





Theorieopdracht





Theorieopdracht

Maak gebruik van de demo code i.v.m. de APA102C LED's om enkele vragen te beantwoorden.

De code heeft als naam: 'Nucleo-L476RG - DMA - memory to SPI - APA102C'.

Zie: "**Theorieopdracht 5 – SPI en APA102C**".